CONCLUSÕES

Carregamento do Dataset e Análise de Integridade

O dataset *'Recyclable and Household Waste Classification'* foi carregado com sucesso. Ele continha 15.000 imagens - com 30 classes e 500 imagens cada-, todas no formato PNG e com dimensões de 256x256 pixels, 3 canais de cor (RGB). Não foram encontradas imagens corrompidas no dataset inicial.

```
print(f'Total de Imagens: {len(df)}')
print(f'Shape do DataFrame: {df.shape[0]} linhas, {df.shape[1]} colunas')

Total de Imagens: 15000
Shape do DataFrame: 15000 linhas, 8 colunas
```

	classe	quantidade
0	plastic_shopping_bags	500
1	cardboard_packaging	500
2	disposable_plastic_cutlery	500
3	styrofoam_food_containers	500
4	glass_food_jars	500

```
format_counts = Counter(formats)

print("Formatos encontrados no dataset:")
print(format_counts)

Formatos encontrados no dataset:
Counter({'PNG': 15000})
```

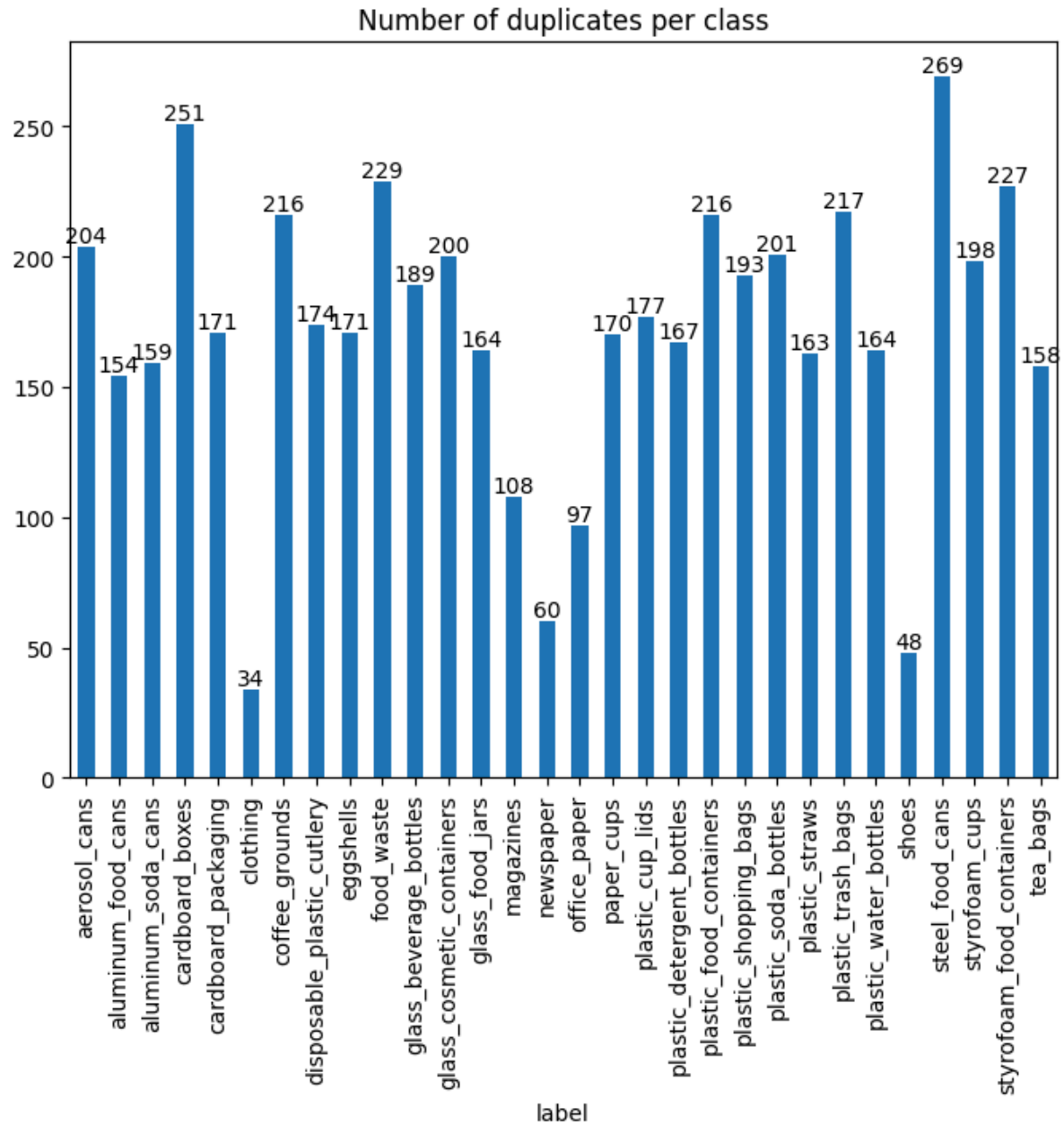
```
df.describe().loc[['mean', 'std', 'min', 'max']]
```

	width	height	channels
mean	256.0	256.0	3.0
std	0.0	0.0	0.0
min	256.0	256.0	3.0
max	256.0	256.0	3.0

```
print(f'Total de Imagens Corrompidas: {count_corrupted}\nImagens corrompidas {corrupted}')

Total de Imagens Corrompidas: 0
Imagens corrompidas []
```

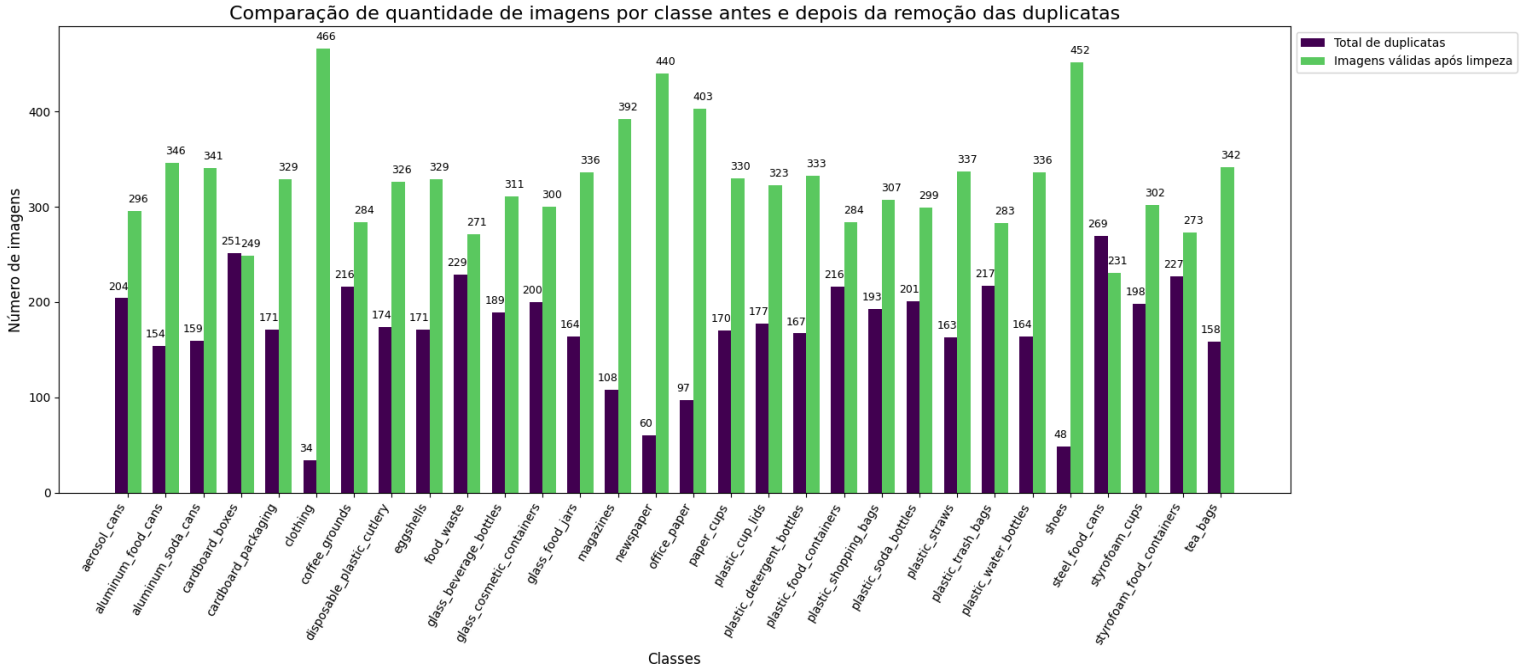
Análise de Duplicatas

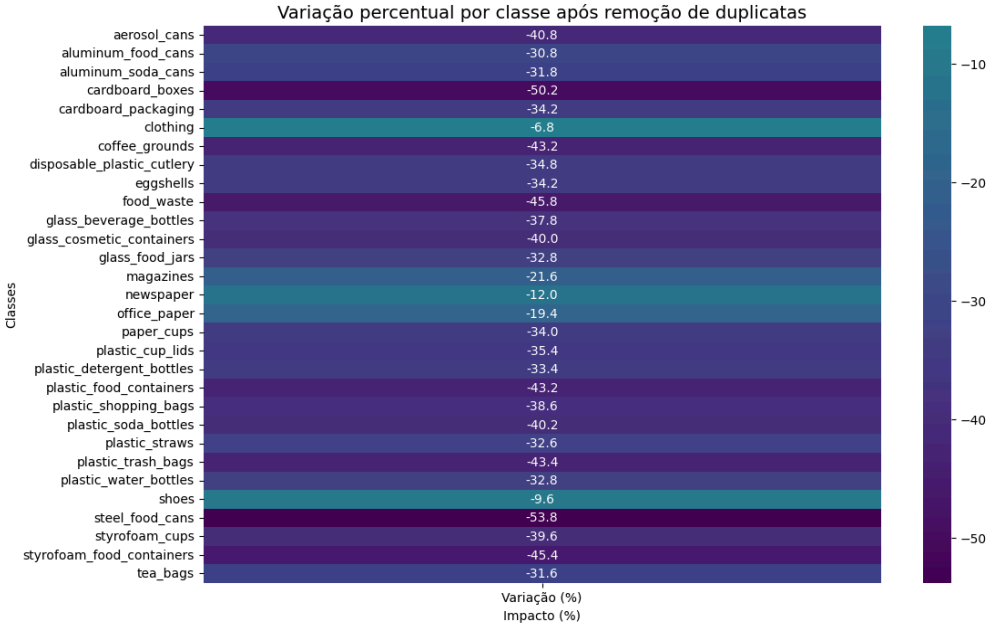
Foi identificada uma grande quantidade de imagens duplicadas (5.149) usando o método de hashing de imagens (imagehash.phash). As classes com os maiores números de duplicatas foram: steel_food_cans (269), cardboard_boxes (251) e food_waste (229). A remoção dessas duplicatas exatas resultou em um dataset limpo com 9.851 imagens.



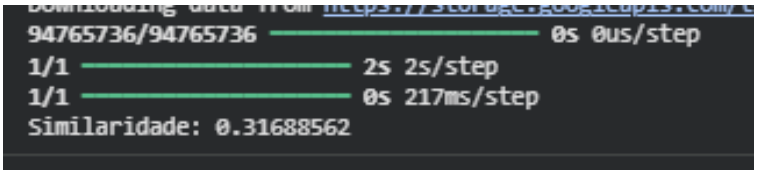

```
df_clean = df.drop_duplicates(subset=['image_hash'], keep='first')

print(f"Imagens originais: {len(df)}")
print(f"Imagens após remoção de duplicatas: {len(df_clean)}")

Imagens originais: 15000
Imagens após remoção de duplicatas: 9851
```

A análise do impacto da remoção de duplicatas mostrou que algumas classes foram significativamente afetadas (por exemplo, 'aluminum_food_cans' com -51,6% de variação), enquanto outras tiveram um impacto menor, indicando uma melhora na qualidade e representatividade do dataset, evitando viés de treinamento.





A verificação de quase duplicatas usando embeddings de ResNet50 e cálculo de similaridade cosseno mostrou que as imagens com similaridade moderada não foram removidas, pois ainda contribuem com informações únicas.



Preparação dos Dados para Treinamento

- Foram selecionadas as 10 classes com maior quantidade de imagens após a limpeza de duplicatas, resultando em um dataset final com 3.855 imagens.

```
Total de imagens depois: 3855

Top 10 classes mantidas e suas quantidades:
labels
clothing                 466
shoes                    452
newspaper                440
office_paper             403
magazines                392
aluminum_food_cans       346
tea_bags                 342
aluminum_soda_cans       341
plastic_straws           337
plastic_water_bottles    336
Name: count, dtype: int64
```

- O dataset foi dividido em conjuntos de treino (70%), validação (15%) e teste (15%) de forma estratificada para garantir a representatividade das classes em cada conjunto.
- Foram configurados geradores de imagens com data augmentation (rotação, deslocamento, inversão horizontal) para o conjunto de treino e apenas pré-processamento para validação e teste, utilizando a função preprocess_input da ResNet50.

Treinamento do Modelo ResNet50:

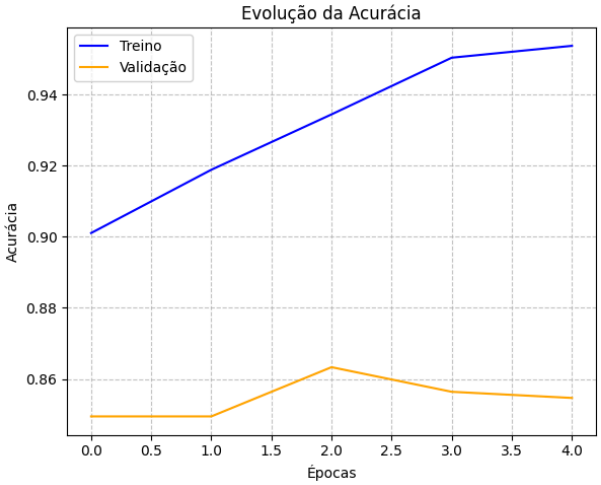
- Um modelo ResNet50 pré-treinado na base de dados ImageNet foi utilizado como base (base_model).
- As camadas do base_model foram congeladas (trainable = False) para aproveitar os recursos já aprendidos.
- Camadas adicionais (GlobalAveragePooling2D e duas camadas Dense) foram adicionadas para adaptar o modelo à tarefa de classificação das 10 classes de resíduos.
- O modelo foi compilado com o otimizador Adam (taxa de aprendizado 0.001), função de perda 'categorical_crossentropy' e métrica 'accuracy'.
- O treinamento foi realizado por 5 épocas com early stopping (patience=2) monitorando a perda de validação. O melhor epoch foi o 3.

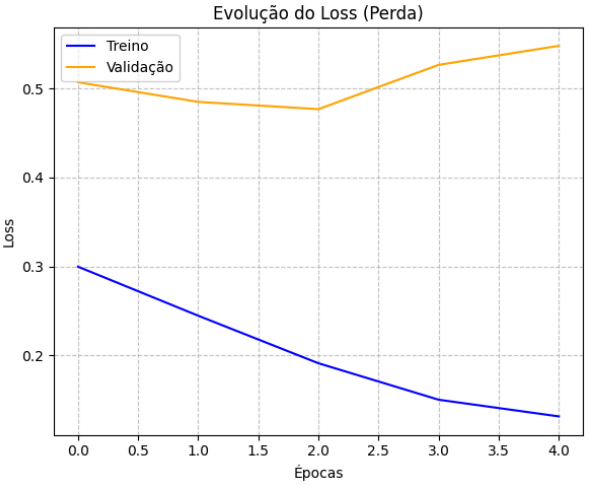
```
Iniciando o treinamento...
Epoch 1/10
85/85 ━━━━━━━━━━━━━━━━━━━━ 613s 7s/step - accuracy: 0.9147 - loss: 0.2729 - val_accuracy: 0.8495 - val_loss: 0.5072
Epoch 2/10
85/85 ━━━━━━━━━━━━━━━━━━━━ 643s 8s/step - accuracy: 0.9235 - loss: 0.2270 - val_accuracy: 0.8495 - val_loss: 0.4851
Epoch 3/10
85/85 ━━━━━━━━━━━━━━━━━━━━ 643s 8s/step - accuracy: 0.9324 - loss: 0.1856 - val_accuracy: 0.8633 - val_loss: 0.4770
Epoch 4/10
85/85 ━━━━━━━━━━━━━━━━━━━━ 597s 7s/step - accuracy: 0.9567 - loss: 0.1343 - val_accuracy: 0.8564 - val_loss: 0.5268
Epoch 5/10
85/85 ━━━━━━━━━━━━━━━━━━━━ 599s 7s/step - accuracy: 0.9466 - loss: 0.1439 - val_accuracy: 0.8547 - val_loss: 0.5482
```

Avaliação do Modelo

O modelo treinado alcançou uma acurácia ponderada de 0.87 no conjunto de teste.
O relatório de classificação (classification_report) mostrou bom desempenho (precision, recall, f1-score) para a maioria das classes, com algumas como 'plastic_water_bottles' atingindo f1-score de 0.96 e 'shoes' com 0.94.

```
--- Relatório de Classificação ---
                      precision    recall  f1-score   support

   aluminum_food_cans       0.81      0.83      0.82        52
   aluminum_soda_cans       0.82      0.80      0.81        51
             clothing       0.89      0.89      0.89        70
            magazines       0.91      0.88      0.90        59
            newspaper       0.83      0.83      0.83        66
         office_paper       0.83      0.79      0.81        61
       plastic_straws       0.84      0.92      0.88        50
plastic_water_bottles       0.98      0.94      0.96        51
                shoes       0.92      0.96      0.94        68
             tea_bags       0.90      0.88      0.89        51

             accuracy                           0.87       579
            macro avg       0.87      0.87      0.87       579
         weighted avg       0.87      0.87      0.87       579

CPU times: user 1h 24min 5s, sys: 5min 28s, total: 1h 29min 33s
Wall time: 53min 17s
```





Contribuições do Estudo

- *Metodologia de Limpeza de Dados*: Demonstração prática da importância da detecção e remoção de imagens duplicadas para garantir a qualidade do dataset e evitar viés no treinamento de modelos de aprendizado de máquina.

- *Aplicação de Transfer Learning*: Utilização eficaz da arquitetura ResNet50 pré-treinada para uma tarefa de classificação de resíduos, mostrando como o transfer learning pode acelerar o desenvolvimento e melhorar o desempenho em problemas com datasets menores.

- *Classificação de Resíduos*: O modelo desenvolvido é capaz de classificar diferentes tipos de resíduos recicláveis e domésticos com boa acurácia, o que tem potencial para aplicações em sistemas de separação automatizada de lixo.

Possibilidades de Novos Estudos

- *Otimização do Modelo*: Explorar o ajuste fino (fine-tuning) das camadas do base_model da ResNet50 (descongelando algumas camadas e treinando com uma taxa de aprendizado menor) para potencialmente melhorar ainda mais a acurácia.

- *Dataset Expandido*: Incluir mais classes de resíduos ou expandir o dataset existente com mais imagens únicas para aumentar a robustez do modelo.

- *Testes com Quase Duplicatas*: Investigar o impacto de quase duplicatas com diferentes limiares de similaridade. Poderiam ser feitas comparações entre modelos treinados com e sem a remoção dessas quase duplicatas para verificar se há overfitting.

- *Outras Arquiteturas*: Comparar o desempenho da ResNet50 com outras arquiteturas de redes neurais convolucionais (ex: VGG, Inception, EfficientNet) para encontrar a mais eficiente e precisa para essa tarefa.

- *Deployment*: Desenvolver uma aplicação (web ou mobile) que utilize o modelo treinado para permitir que usuários classifiquem resíduos em tempo real.

- *Análise de Viés*: Aprofundar a análise de viés do modelo para garantir que ele não esteja aprendendo associações indesejadas ou discriminatórias em relação a certos tipos de resíduos ou condições de imagem.

- *Interpretabilidade do Modelo*: Utilizar técnicas de interpretabilidade (ex: Grad-CAM) para entender quais partes das imagens o modelo está usando para fazer suas previsões.

Informações Complementares:

- *A escolha do random_state=42 em train_test_split garante a reprodutibilidade da divisão dos dados.*
- *O ImageDataGenerator com data augmentation é crucial para aumentar a variabilidade do conjunto de treino e reduzir o overfitting.*
- *O EarlyStopping ajuda a prevenir o overfitting e economizar recursos de treinamento, parando o processo quando a performance na validação não melhora mais.*